NinjaRS benchmark

Batched one-loop integrand reduction in Rust versus MadLoop (Ninja + OneLOop)

2026-10-19 · AMD EPYC 9754 (shared host), all timings pinned to one core

Reading guide. Every multiplier is *NinjaRS time ÷ reference time*, and each cell names its reference. In Table 1 both codes evaluate the same diagram, helicity configuration and phase-space points, but MadLoop runs its own pipeline on its own inputs: its own loop numerator and propagator order, and the rank it declares (3 instead of 2 for the triangles, 5 instead of 4 for the pentagon). Generation is compared with MadLoop output plus compilation, the numerator with MadLoop's loop-coefficient computation and Ninja interface (including Ninja's tensor-numerator expansions), the reduction with original Ninja inside MadLoop, the master integrals with OneLOop and the total with the complete MadLoop pipeline. In Table 2 both codes receive identical inputs at the same rank and are compared with original Ninja and OneLOop component by component. Green cells mean NinjaRS is faster, red slower; white is parity. NinjaRS evaluates batches of 32 phase-space points; times are per point. At numerator rank $n+2$ (Section 5), beyond original Ninja, the reference is the tensor-integral library COLLIER, a different method.

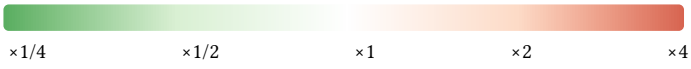
Performance gates (binary64, NinjaRS batched; ranges over the physical diagrams of Table 1, identical-input values from Table 2)

(a) Pure reduction	NinjaRS ≤ original Ninja on identical inputs	**×0.49–0.59**
(b) Master integrals	oneloopmaster native ≤ 1.3 × OneLOop on the same calls	**×0.27–0.61**
(c) End to end	NinjaRS (SymJIT O2 numerator + reduction + masters) ≤ MadLoop pipeline	**×0.17–0.54**

1 Summary of all test cases

Table 1: **Physical diagrams versus the MadLoop pipeline.** Generation time and per-phase-space-point evaluation time (binary64, one helicity configuration, a single loop diagram). Each cell shows the NinjaRS value in bold, the reference value below it and the multiplier NinjaRS ÷ reference. Generation: NinjaRS CLI graph preparation, coefficient planning and SymJIT O2 compilation versus MadLoop process output plus compilation (ML). Numerator: SymJIT O2 coefficient heads and tensor-numerator expansions (with propagator and S-matrix preparation) versus MadLoop's loop-coefficient construction, its Ninja interface (tensor copy, momentum inversion, S-matrix) and Ninja's tensor-numerator expansions (ML). Reduction: pure reduction versus original Ninja's reduction inside MadLoop, i.e. Ninja's time minus its tensor-numerator callbacks and master integrals, which includes its per-call setup (N). Masters: oneloopmaster's native code versus OneLOop (O). Total: complete evaluation versus the complete MadLoop pipeline (ML). The rank column gives NinjaRS's rank and, in brackets, the rank MadLoop passes to Ninja.

Test case	n / r	Generation [s]	Numerator [µs]	Reduction [µs]	Masters [µs]	Total [µs]
gg → H top triangle	3 / 2 (3)	**0.129** ML 4.26 ×0.03	**0.965** ML 15.2 ×0.06	**1.29** N 3.42 ×0.38	**1.92** O 3.33 ×0.58	**4.22** ML 21.7 ×0.19
γγ → γγ top box	4 / 4 (4)	**5.35** ML 5.19 ×1.03	**20.1** ML 41.5 ×0.48	**5.30** N 10.6 ×0.50	**5.53** O 9.27 ×0.60	**32.7** ML 60.5 ×0.54
gg → HH top triangle	3 / 2 (3)	**0.150** ML 4.57 ×0.03	**0.930** ML 13.4 ×0.07	**1.28** N 3.36 ×0.38	**0.864** O 1.96 ×0.44	**3.08** ML 18.4 ×0.17
gg → HH top box	4 / 4 (4)	**0.689** ML 4.95 ×0.14	**11.2** ML 36.3 ×0.31	**5.31** N 10.8 ×0.49	**13.4** O 33.9 ×0.39	**30.4** ML 80.8 ×0.38
gg → HHH top triangle	3 / 2 (3)	**0.150** ML 5.66 ×0.03	**1.04** ML 13.2 ×0.08	**1.28** N 3.32 ×0.39	**0.783** O 1.25 ×0.63	**3.11** ML 17.7 ×0.18
gg → HHH top box	4 / 4 (4)	**1.58** ML 6.04 ×0.26	**11.2** ML 36.2 ×0.31	**5.58** N 10.6 ×0.53	**13.2** O 28.2 ×0.47	**30.6** ML 74.4 ×0.41
gg → HHH top pentagon	5 / 4 (5)	**3.04** ML 6.83 ×0.44	**15.0** ML 105 ×0.14	**8.99** N 26.5 ×0.34	**54.3** O 110 ×0.49	**79.4** ML 241 ×0.33



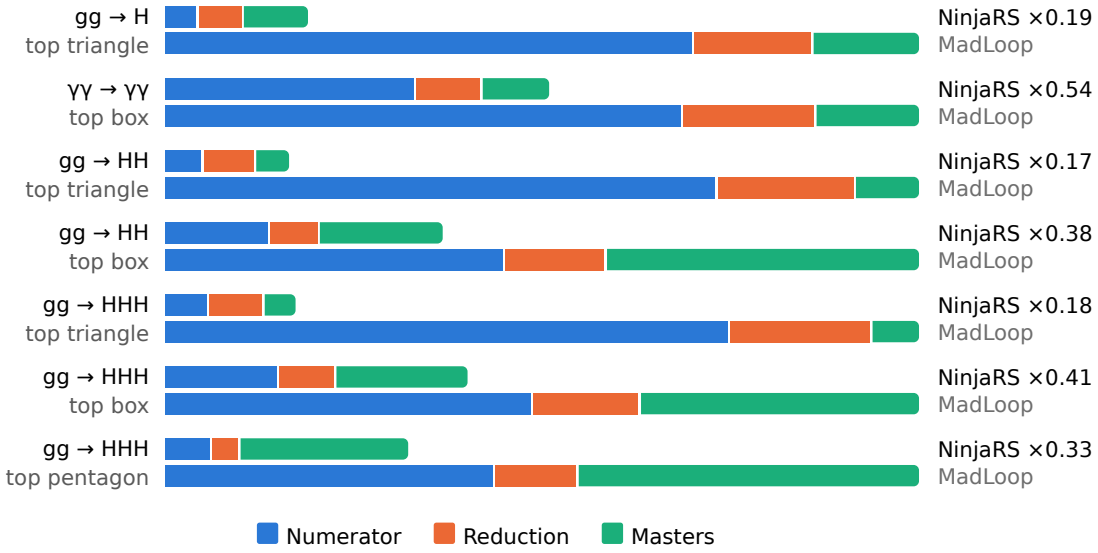


Figure 1: Per-point evaluation time of each diagram, NinjaRS (upper bar, batched) and the MadLoop pipeline (lower bar), stacked by component and drawn to the scale of the measured MadLoop total. Components are timed separately, so a stack can differ from its measured total by a few per cent. The label gives NinjaRS's measured total as a multiple of MadLoop's; Table 1 lists the values.

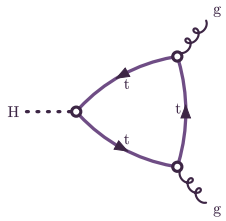
Table 2: **Identical inputs versus original Ninja and OneLOop.** Both implementations receive the same tensor-numerator coefficients, propagators and invariants (binary64), and evaluate the same rank. Tensor numerator: Ninja-layout tensor expansions in NinjaRS's batched `TensorNumerator` versus Ninja's `TensorNumerator` (N); pure reduction with replayed numerator and master values (N); master integrals on the identical call stream, oneloopmaster native versus OneLOop (O); total with live numerator and masters. Original Ninja is built with `g++ -O3 -fcx-fortran-rules` (MadLoop's flag; 20 % faster than plain `-O3`). The last column is the maximal difference of the three Laurent coefficients relative to the largest, NinjaRS versus Ninja on these identical inputs, over the three points; † marks single ill-conditioned points where neither code is accurate in binary64 (see Section 3).

Case	n / r	Tensor num. [µs]	Reduction [µs]	Masters [µs]	Total [µs]	Δ vs Ninja
gg → H, top triangle	3 / 2	0.437 N 0.934 ×0.47	1.27 N 2.60 ×0.49	1.77 O 6.53 ×0.27	3.54 N+O 10.0 ×0.35	1.3e−14
γγ → γγ, top box	4 / 4	9.49 N 22.8 ×0.42	5.24 N 9.19 ×0.57	5.56 O 10.1 ×0.55	20.4 N+O 43.0 ×0.47	9.7e−13
gg → HH, top triangle	3 / 2	0.427 N 0.931 ×0.46	1.29 N 2.54 ×0.51	0.863 O 1.90 ×0.45	2.59 N+O 5.41 ×0.48	3.4e−15
gg → HH, top box	4 / 4	9.57 N 23.0 ×0.42	5.50 N 9.28 ×0.59	13.7 O 27.8 ×0.49	28.8 N+O 61.3 ×0.47	6.9e−13
gg → HHH, top triangle	3 / 2	0.428 N 0.921 ×0.46	1.28 N 2.54 ×0.50	0.771 O 1.27 ×0.61	2.47 N+O 4.75 ×0.52	9.1e−16
gg → HHH, top box	4 / 4	9.34 N 22.7 ×0.41	5.36 N 9.45 ×0.57	12.7 O 22.5 ×0.57	27.6 N+O 55.8 ×0.49	1.3e−14
gg → HHH, top pentagon	5 / 4	11.4 N 22.6 ×0.50	9.13 N 18.6 ×0.49	55.0 O 98.2 ×0.56	75.5 N+O 141 ×0.53	1.1e−14
Synthetic topologies (random kinematics, real and complex masses, dense random tensors)						
`syn-gghhh-gHgHH-n5-r4`	5 / 4	11.7 N 22.6 ×0.51	9.00 N 18.6 ×0.48	28.2 O 64.2 ×0.44	50.1 N+O 108 ×0.46	1.3e−12
`syn-gghhh-gHgHH-n5-r5`	5 / 5	36.5 N 81.4 ×0.45	13.1 N 22.7 ×0.58	27.7 O 62.3 ×0.44	78.6 N+O 169 ×0.46	1.4e−11
`syn-gghhh-ggHHH-n5-r4`	5 / 4	12.0 N 22.7 ×0.53	9.20 N 18.5 ×0.50	62.1 O 117 ×0.53	83.0 N+O 162 ×0.51	2.9e−12
`syn-gghhh-ggHHH-n5-r5`	5 / 5	36.6 N 81.6 ×0.45	13.2 N 23.8 ×0.55	63.1 O 118 ×0.53	113 N+O 228 ×0.50	5.0e−4†
`syn-gghhh-nosmat-n5-r4`	5 / 4	11.2 N 22.7 ×0.50	9.11 N 18.6 ×0.49	61.0 O 118 ×0.52	81.9 N+O 161 ×0.51	3.7e−12
`syn-n2-r1`	2 / 1	0.033 N 0.059 ×0.56	0.258 N 0.430 ×0.60	0.162 O 0.207 ×0.79	0.455 N+O 0.673 ×0.68	4.0e−15
`syn-n2-r2`	2 / 2	0.199 N 0.428 ×0.46	0.290 N 0.483 ×0.60	0.167 O 0.199 ×0.84	0.657 N+O 1.10 ×0.60	1.9e−13
`syn-n2-r3`	2 / 3	1.04 N 2.73 ×0.38	0.514 N 0.984 ×0.52	0.168 O 0.295 ×0.57	1.77 N+O 4.08 ×0.43	3.5e−11

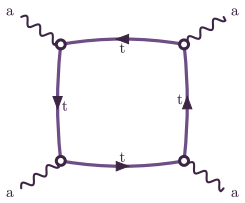
Case	n / r	Tensor num. [µs]	Reduction [µs]	Masters [µs]	Total [µs]	Δ vs Ninja
syn-n3-r2	3 / 2	0.462 N 0.931 ×0.50	1.27 N 2.54 ×0.50	1.84 O 4.27 ×0.43	3.59 N+O 7.75 ×0.46	8.3e−14
syn-n3-r3	3 / 3	1.90 N 4.68 ×0.41	1.74 N 2.97 ×0.59	2.14 O 3.87 ×0.55	5.64 N+O 11.7 ×0.48	5.5e−14
syn-n3-r4	3 / 4	7.97 N 20.4 ×0.39	4.02 N 7.20 ×0.56	1.59 O 5.08 ×0.31	13.5 N+O 32.9 ×0.41	7.2e−14
syn-n4-r3	4 / 3	2.56 N 5.37 ×0.48	3.50 N 7.19 ×0.49	11.9 O 16.9 ×0.70	18.1 N+O 29.9 ×0.60	2.4e−12
syn-n4-r4	4 / 4	9.60 N 22.6 ×0.42	5.09 N 8.75 ×0.58	13.7 O 15.9 ×0.87	29.2 N+O 48.1 ×0.61	2.6e−13
syn-n4-r5	4 / 5	33.2 N 80.4 ×0.41	11.1 N 19.9 ×0.56	10.6 O 12.2 ×0.87	55.0 N+O 114 ×0.48	4.0e−12
syn-n5-r4	5 / 4	11.6 N 22.7 ×0.51	9.08 N 18.6 ×0.49	36.9 O 68.7 ×0.54	57.1 N+O 113 ×0.51	2.1e−13
syn-n5-r5	5 / 5	36.7 N 81.4 ×0.45	13.5 N 22.6 ×0.60	38.2 O 75.4 ×0.51	89.4 N+O 184 ×0.49	7.4e−12
syn-n5-r6	5 / 6	111 N 260 ×0.43	29.6 N 50.9 ×0.58	47.6 O 91.9 ×0.52	192 N+O 411 ×0.47	2.6e−9
syn-n6-r5	6 / 5	40.4 N 74.9 ×0.54	19.1 N 37.6 ×0.51	321 O 495 ×0.65	385 N+O 614 ×0.63	3.7e−8
syn-n6-r6	6 / 6	115 N 240 ×0.48	27.6 N 46.4 ×0.59	271 O 428 ×0.63	416 N+O 731 ×0.57	3.6e−2†
syn-n6-r7	6 / 7	315 N 695 ×0.45	57.1 N 97.2 ×0.59	333 O 541 ×0.62	743 N+O 1360 ×0.55	1.7e−7

2 The benchmarked diagrams

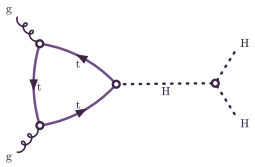
The physical diagrams are drawn from the same FeynKit DOT files that NinjaRS prepares (`ninjars draw`, laid out and styled by GammaLoop's Linnest package): gluons as coils, photons as waves, top quarks as solid lines with fermion-flow arrows and Higgs bosons dashed. Each diagram is matched one to one with the MadLoop loop diagram given by its loop-filter identifier.



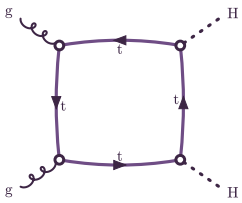
gg → H
$n = 3$, $r = 2$ (3), ML #3



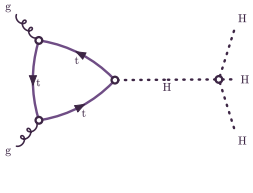
γγ → γγ
$n = 4$, $r = 4$ (4), ML #182



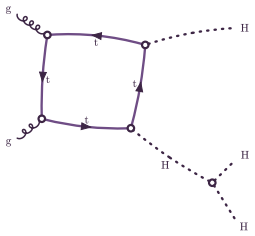
gg → HH
$n = 3$, $r = 2$ (3), ML #11



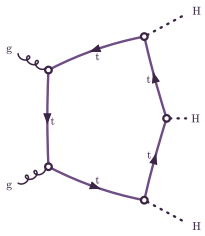
gg → HH
$n = 4$, $r = 4$ (4), ML #10



gg → HHH
$n = 3$, $r = 2$ (3), ML #66

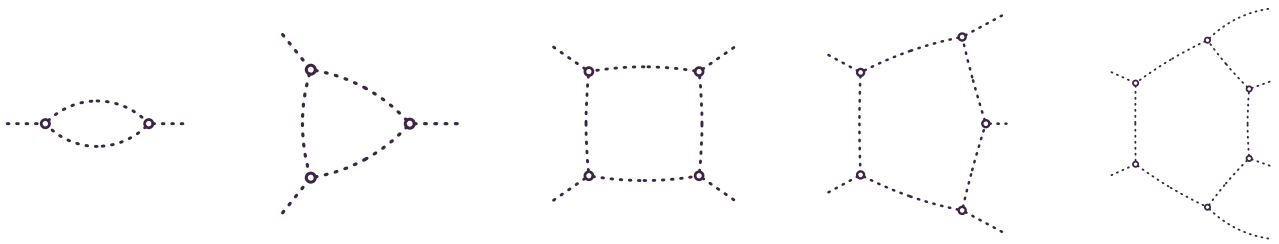


gg → HHH
$n = 4$, $r = 4$ (4), ML #58



gg → HHH
$n = 5$, $r = 4$ (5), ML #55

Figure 2: The seven physical one-loop diagrams with a top-quark loop. Below each drawing: number of propagators n, numerator rank r in NinjaRS (rank passed to Ninja by MadLoop in brackets) and the MadLoop loop-diagram identifier.



$n = 2$ ranks 1–3 | $n = 3$ ranks 2–4 | $n = 4$ ranks 3–5 | $n = 5$ ranks 4–6 | $n = 6$ ranks 5–7

Figure 3: Scalar topologies of the synthetic cases ($n = 2, \ldots, 6$). Each synthetic case attaches a dense random tensor numerator of rank $n - 1$, n or $n + 1$, random real external kinematics with time-like and space-like invariants, and real or complex internal masses to one of these topologies.

3 Correctness

Speed is only meaningful at equal accuracy. The binary64 results of the benchmark were checked as follows.

- **Reduction.** With identical master values, `ninjars::fast` agrees with original Ninja to 7×10^{-13} (relative to the largest modulus among Ninja's Laurent coefficients and its cut-constructible and rational parts) on every phase-space point of the physical cases and to 4×10^{-12} on 30 dedicated edge cases (light-like and zero invariants, mixed and complex masses, infrared-divergent triangles and boxes, all tadpole reference bases, ranks $n - 5, \ldots, n + 1$ and $n = 7$), including the split into cut-constructible and rational parts. Where the two differ more (the synthetic pentagons and hexagons `syn-n5-r6`, `syn-n6-r5` and `syn-n6-r7` at 2×10^{-9} to 1.1×10^{-7}, and the ill-conditioned point of `syn-gghhh-ggHHH-n5-r5` discussed below), both are compared with the generic NinjaRS engine at 256 bits: over these and the edge-case points (42 in total) the error ratio NinjaRS ÷ Ninja has median 1.1, i.e. the deviation is inherited binary64 conditioning, not a porting error. Results are bitwise identical for every batch width and lane order, and the warmed path performs no heap allocation.
- **End to end.** On identical inputs every physical diagram matches original Ninja to 10^{-12} or better (last column of Table 2), and the complete NinjaRS pipeline, with its own SymJIT numerator, matches Ninja fed with the exported tensors to 2×10^{-12}. Original Ninja gives the same result on MadLoop's own inputs as on the NinjaRS-exported tensors up to one constant factor at all three points (-2 for the QCD diagrams, $-\frac{1}{3}$ for $\gamma\gamma \to \gamma\gamma$, spread $\leq 1.2 \times 10^{-14}$). Together with a diagram check and identical propagator invariants and masses, this fixes the diagram and its helicity configuration; the propagator order differs (MadLoop's is cyclic).
- **Masters.** On all 1,998 master calls that original Ninja makes in the benchmark (34 call streams: the physical diagrams as exported by NinjaRS and by MadLoop, and the synthetic topologies), oneloopmaster's native code agrees with OneLOop 3.6 to 8×10^{-14} on the physical diagrams and to 3.6×10^{-11} on the synthetic ones (the worst a D_0 near a vanishing Gram determinant, discussed below), except for one rank-3 tensor bubble (2.5×10^{-9}). The native bubble tensors were validated against 595 direct Feynman-parameter integrations, including thresholds and massless limits.

Ill-conditioned lanes. Two synthetic cases differ from Ninja by more than 10^{-6} (marked † in Table 2). In each, one lane's finite part is a tiny remainder of cancelling master-integral contributions. The exact values were computed with the generic NinjaRS engine at 256–512 bits, where the evaluations agree to 40 digits (`syn-n6-r6`) and to binary64 rounding (`syn-gghhh-ggHHH-n5-r5`); the per-point data are in `validation/benchmark/ill-conditioned/`. Against them, neither binary64 code is accurate on these lanes:

- `syn-gghhh-ggHHH-n5-r5`, lane 1, borders a box with normalized Gram determinant 8×10^{-8}. Ninja is off by 3.0×10^{-4} and NinjaRS by 4.6×10^{-4}; with exact masters rounded to binary64 NinjaRS is off by 3.1×10^{-4}, so most of the loss is in the binary64 reduction itself, in both codes. The rest comes from oneloopmaster's binary64 D_0 at that box, off by 3.6×10^{-11} against OneLOop's 7.9×10^{-15}; it was off by 4.4×10^{-12} (NinjaRS's lane by 3.4×10^{-4}) before oneloopmaster took the contour box's edge roots from exact Källén functions (9c5490b). This regression has been reported upstream and is still open (09b2c74 returns the same D_0).
- `syn-n6-r6`, lane 0, cancels by a factor of about 10^{12}. Ninja is off by 2.3 % and NinjaRS by 1.3 % (1.2 % with Ninja's own master values). NinjaRS was off by 24 % before oneloopmaster conditioned the dilogarithm differences of nearly coincident triangle roots: the triangle $C_0(-237088.3, -236743.3, 0; 29929 - 259.5i, 0, 6464.16 - 168.84i)$ was off

by 2.4×10^{-13} against OneLOop's 3.3×10^{-15} and is now off by 2.2×10^{-15}, with the divided-difference expansion OneLOop uses for such roots.

Neither code's stability checks flag these lanes. All other lanes of these cases agree with the exact values to 10^{-9} or better.

Defects found on the way. The comparison exposed and fixed the following defects.

- **Symbolica** (binary64 only): the dilogarithm went through an MPFR callback (12 µs to 2 ms per call, making native C_0/D_0 25–430× slower than OneLOop); a binary64 series kernel now serves it (PR #89), and oneloopmaster's binary64 code calls it directly.
- **oneloopmaster** (fixed in the exact expressions, so every route — native, hand-written, SymJIT, interpreter, arbitrary precision and Dual numbers — carries the fix, except where noted; NinjaRS pins oneloopmaster 09b2c74, and the last eight items and the full extent of the B_{1111} error were found after the previous revision of this report, which used 264a6ce):
  - $B_0/B_{0'}$ returning NaN with a massless line;
  - inexact unit roots in massless bubble tensors (fixed in the native port of OneLOop's bubble routines), and wrong lips at $p^2 = 0$ with masses of opposite sign;
  - a sign error in OneLOop's own B_{1111}: its mass-swap reconstruction is negated in all three swap branches, not only at $p^2 = 0$ (the native rank-four bubble family, added for numerator rank $n + 2$, uses the corrected one);
  - wrong lips in two- and three-mass and 't Hooft–Veltman triangles;
  - errors in infrared boxes and in four-mass boxes with $s_{12}s_{23} < 0$;
  - in real-mass contour boxes, analytically real logarithms whose lip was set by rounding (off by multiples of $2\pi i$ on about a quarter of such points). OneLOop 3.6 itself departs from its own vanishing-width limit at some of these points;
  - a box near a vanishing Gram determinant (accuracy 7×10^{-10}, then 4×10^{-12}; 3.6×10^{-11} in binary64 since the exact Källén edge roots below, a regression reported upstream and still open);
  - exact-zero branch tests decided by rounding noise in arbitrary precision (C_0 wrong at 256, 512 and 1024 bits for some arguments);
  - an expression-construction blow-up introduced and removed during these fixes (shared definitions built in 80 s with 16 GB of memory; now 59 MB, built in 0.35 s);
  - dilogarithm differences of nearly coincident triangle roots cancelling in binary64 (the C_0 of `syn-n6-r6` above), now expanded as OneLOop does; and box coefficients formed by cancellation, which cost bits in arbitrary precision (a D_0 with an invariant of 2^{-114} kept 7 to 55 correct bits at 132 to 184 bits);
  - real-mass three-mass boxes in OneLOop's `boxf33` chart, wrong in some propagator labellings (in 39 of 154 boxes of a sweep, OneLOop 3.6 included); they now take the contour box in every labelling;
  - contour boxes losing 8 to 10 digits in binary64 next to a pseudo-threshold beside a massless line, and up to 5.7×10^{-4} from far real roots of an edge quadratic; the edge roots now come from exact Källén functions and far roots from a series in $1/z$;
  - real four-mass boxes in OneLOop's `boxf4` chart, wrong in some labellings (OneLOop 3.6 in 118 of 6,040 relabellings of a sweep); they now use eq. (3.61) of Denner and Dittmaier in a labelling that meets its conditions, with exact branch decisions, and agree in every labelling of the sweeps;
  - one- and two-mass boxes next to collinear limits (up to 2.9×10^{-7} off in binary64, as in OneLOop's double precision, and $O(1)$ at invariants of 10^{-14}); now within 10^{-11} on a sweep of 400 such boxes;
  - massless boxes next to collinear limits, and massless, one- and two-mass boxes whose discriminant also nearly vanishes there (up to 2×10^{-2} off in binary64);
  - in the native double-double code only, products whose low word depended on the operand order, which let exact branch tests read a 10^{-35} residue as a sign: an equal-mass four-mass D_0 lost its real part in one of eight labellings (in NinjaRS a 12-point double-double amplitude was off by 1.6×10^{-3}). The native products are now symmetric; the interpreter's double-double route (tensor families, Dual numbers) keeps Symbolica's product;
  - in the interpreter only, the cancellation test of the conditioned dilogarithm differences above evaluating both dilogarithms two or three times, slowing the arbitrary-precision runs of Section 4 by 1.4–1.5×; they are now evaluated once (09b2c74).
- **NinjaRS:**
  - rounded light-like invariants entering the masters without OneLOop's on-shell threshold (a 10^{-7} error in $\gamma\gamma \to \gamma\gamma$ below threshold);

  - in arbitrary precision, invariants differenced from rounded momenta (the four-mass D_0 was wrong by orders of magnitude at 128 bits while reporting 8 correct bits); they are now derived exactly from the prepared momenta;
  - a degenerate rational bubble reference vector (an $O(10^{13})$ blow-up at exactly coplanar rational kinematics);
  - bubble reference vectors tied in score, chosen by rounding: at some precisions this moved ± 1375 between the cut-constructible and rational parts while their sum stayed exact;
  - inaccurate Passarino–Veltman bubble tensors in an intermediate provider.
- **OneLOop reference copies:** two wrong D_0 values, in versions 3.4 and a 2024 snapshot. OneLOop 3.6 and COLLIER agree with each other there.

Apart from the on-shell threshold, none of these defects affected the accuracy of the physical benchmark diagrams in binary64; the dilogarithm affected only their speed. The oneloopmaster changes since 264a6ce move the binary64 results of the physical diagrams by at most 6×10^{-15}.

4 Arbitrary precision end to end

The same CLI evaluates a physical diagram at any working precision, from the DOT graph to the integrated result, without a separate quadruple-precision build. For the $gg \to HHH$ top pentagon (graph 26) at its three benchmark points,

```
ninjars reduce examples/graphs/physical/gg-hhh/top-pentagon-26-pool.json \
                        --precision float --bits 700
```

prepares the numerator exactly (FeynKit, exact radical momenta and polarization vectors), runs the generic reduction engine on Symbolica's arbitrary-precision `Float` (cut solutions, Laurent divisions and subtractions at 700 bits) and evaluates oneloopmaster's exact master expressions at the same precision. All three points take 7.7–7.8 seconds in total at 700 bits and 8.7–9.0 seconds at 1000 bits, including the one-time preparation. The same CLI built on the previous pin, oneloopmaster 149ddff, run alternately with it on the same core (CPU 245), takes 10.5 and 13.3–13.7 seconds: oneloopmaster 3cbe08d, which conditions the dilogarithm differences, had made the run about 1.4× (700 bits) and 1.5× (1000 bits) slower, and 09b2c74 fixes this (a CLI built on 264a6ce, before 3cbe08d, takes 7.3–7.4 and 8.5–8.6 seconds there). The 700-bit results agree with a 1000-bit run to 208 digits; the poles and the rational part vanish to working precision (below 10^{-211} at 700 bits and 10^{-301} at 1000 bits; the double pole is exactly zero), as they must for this finite pentagon. The table compares the finite part across precisions. Accuracy is always measured this way, by comparison with a higher precision: the significant precision that Symbolica's `Float` tracks counts cancellations but not accumulated rounding, and overstates the accuracy by a few bits (3–8 bits in the 700-bit runs above, up to 13 bits in precision scans of the $\gamma\gamma \to \gamma\gamma$ boxes).

Table 3: Correct digits of the finite part of graph 26, measured against the 1000-bit evaluation.

$\sqrt{s}$	binary64 (fast engine)	double-double	700 bits
450 GeV	14.3	29.8	208.8
525 GeV	14.0	29.8	209.0
600 GeV	14.7	30.2	208.1

Finite part of graph 26 at $\sqrt{s} = 450$ GeV to 200 significant digits (1000-bit evaluation, normalization of the NinjaRS fixture):

```
Re = 3. 8093488998 4288342115 1261701674 2079345485 6746936859
        0158897991 5031826404 5716902534 1107750350 9959766306
        7939863114 9896847957 1786292361 8269968883 8989360143
        8480205079 7027471359 4284487895 3681501664 366214565
        × 10^-3

Im = 6. 7910272696 2000785356 5042700284 1414347940 5560474076
        6521286244 9465165686 5893536348 9342077051 2294615772
        5743830037 9022950782 5061327204 6646042255 7278710493
        0406715542 2890883288 4027355477 0845854559 187819210
        × 10^-3
```

5 Numerator rank $n + 2$

NinjaRS also reduces numerators of rank $r = n + 2$, one beyond Ninja's higher-rank extension [1] (the algorithm, the new integrand terms and their validation are in `docs/rank-n2.md`). Original Ninja cannot serve as the reference here, so this section compares NinjaRS with the tensor-integral library COLLIER 1.2.10 [2], with its COLI and DD branches [3], [4], [5]. All inputs are the rank $n + 2$ bench cases of `validation/rank-n2`: the 105 suite cases ($n = 1, \ldots, 7$; real, complex, mixed and zero masses; numerators with powers of the integrand μ^2 up to μ^8; 3 phase-space points each) and the six synthetic cases `syn-n1-r3` to `syn-n6-r8`, each lane with a high-precision reference value.

A different method, so a timing reference, not like for like. COLLIER does not reduce the integrand. It computes the tensor integrals $T^{N,\mu_1\cdots\mu_P}$ of every rank $P \le r$ by Passarino–Veltman reduction and, where the Gram or Cayley determinants are small, by expansions in them; the integral of a numerator is then the contraction of these tensor components with the numerator's tensor coefficients. Three consequences shape the comparison:

- **Inputs.** COLLIER takes the tensor in a loop momentum with $p_0 = 0$, and no integrand μ^2. The harness removes μ^2 exactly by pinching ($\mu^2 = q^2 + 2q \cdot V_a + V_a^2 - m_a^2 - D_a$; up to five pieces with fewer propagators per lane) and shifts each piece, in exact rational arithmetic, before any timing; the pieces are then rounded to binary64 once. On the 56 of 111 cases without μ^2 and with $V_0 = 0$, COLLIER receives exactly the tensor NinjaRS receives and no preprocessing is left out of its time.
- **Output.** One COLLIER evaluation gives the finite part for one choice of its regulator parameters Δ_{UV}, $\Delta_{\mathrm{IR}}^{(1)}$, $\Delta_{\mathrm{IR}}^{(2)}$. The single and double poles take two more evaluations with shifted parameters (only the UV pole is returned separately). NinjaRS returns all three Laurent coefficients, and their split into cut-constructible and rational parts, in one evaluation. The tables compare NinjaRS with COLLIER's single evaluation (ε^0, its usual use) and with the three evaluations of the full Laurent expansion.
- **Configuration.** COLLIER 1.2.10 (sha256 `460624ea70921150`…) is built twice in binary64: `precise`, its gfortran Release flags without `-ffast-math`, as every other code in this report; and `stock`, the shipped flags with `-ffast-math`. It runs with its default target accuracy 10^{-8} (`SetReqAcc_cll`), without its global cache, and with the renormalization scale set outside the timed region (`SetMuUV2_cll` costs 1.9 µs, the Δ changes between the three evaluations 5.8 µs, both from DD's determination of the machine precision at every parameter change). DD covers $n \le 4$ at rank $n + 2$ (pentagons up to rank 5, hexagons up to 6). With one integrand per phase-space point, the global cache changes COLLIER's time by ×0.92 on average (×0.51–1.07); the stock build takes ×0.89 of the precise build's time.

NinjaRS runs the fast engine (`ninjars case-bench`: dense `TensorNumerator`, `MuTensorNumerator` with μ^2 blocks, native masters) at batch widths 1 and 32, as in Table 2, and the generic engine in binary64 (`case-bench --engines generic`: `SymbolicaAmplitudeKernels`, native masters and `case-check`'s numerically expanded dense-tensor numerator).

Measurement. Every process is single-threaded and pinned to one core (CPU 260, idle when chosen; the host has no SMT siblings). The 2 rounds run NinjaRS and every COLLIER variant on every case in turn; each quantity is the best of the rounds' medians over blocks of at least 20 ms (9 blocks for NinjaRS, 5 for COLLIER, 5 for the generic engine, which runs once). Other processes took 0.9 % of that core's time during the measurements. The core clock of this shared host varies between 1.8 and 3.1 GHz, so the retired instructions per point (Table 6) are given as a clock-independent check. NinjaRS at `0d0d7e7` (oneloopmaster `09b2c74`), 2026-10-19.

Table 4: **Time per phase-space point at rank $n + 2$, NinjaRS versus COLLIER.** Binary64, one integrand per point, all codes pinned to the same core (CPU 260). NinjaRS fast engine at batch width 32: total in bold and, below, tensor-numerator expansions + pure reduction + masters. COLLIER (precise build): tensor integrals of every piece plus their contraction with the numerator coefficients, for the finite part (ε^0, one evaluation) and for the full Laurent expansion (three evaluations); COLI and, where it applies, DD. Each COLLIER cell gives its time and the multiplier NinjaRS ÷ COLLIER (green: NinjaRS faster). The last column is NinjaRS's generic engine (binary64, width 32) and its multiplier against COLI. `syn-*` are the synthetic cases, the others suite cases without their `rn2-` prefix; `-mu2` and `-muall` carry μ^2 numerators (COLLIER: several pieces); `shifted` has $V_0 \neq 0$.

Case	n / r	NinjaRS fast [µs]	COLI ε^0 [µs]	COLI Laurent [µs]	DD ε^0 [µs]	generic f64 [µs] ÷ COLI
`syn-n1-r3`	1 / 3	0.994 0.744 + 0.145 + 0.103	1.74 ×0.57	11.2 ×0.088	1.82 ×0.55	11.0 ×6.29
`syn-n2-r4`	2 / 4	8.13 6.29 + 1.52 + 0.257	8.61 ×0.94	31.9 ×0.25	7.09 ×1.15	137 ×16
`syn-n3-r5`	3 / 5	61.0 41.7 + 15.5 + 2.36	39.2 ×1.55	124 ×0.49	70.8 ×0.86	707 ×18

Case	n / r	NinjaRS fast [µs]	COLI ε^0 [µs]	COLI Laurent [µs]	DD ε^0 [µs]	generic f64 [µs] ÷ COLI
`syn-n4-r6`	4 / 6	217 157 + 41.9 + 16.9	517 ×0.42	1558 ×0.14	577 ×0.38	2398 ×4.64
`syn-n5-r7`	5 / 7	680 490 + 105 + 73.2	15 070 ×0.045	44 644 ×0.015	—	6610 ×0.44
`syn-n6-r8`	6 / 8	1987 1290 + 194 + 482	87 066 ×0.023	270 632 ×0.0073	—	17 335 ×0.20
`int-complex-n7-r9`	7 / 9	5018 3032 + 344 + 1616	451 285 ×0.011	1 349 050 ×0.0037	—	35 877 ×0.080
`int-zero-n2-r4`	2 / 4	3.30 2.89 + 0.299 + 0.114	7.19 ×0.46	27.5 ×0.12	5.83 ×0.57	—
`irml-zero-n4-r6`	4 / 6	71.4 63.9 + 7.96 + 0.841	483 ×0.15	1459 ×0.049	557 ×0.13	—
`irml-zero-n6-r8`	6 / 8	935 840 + 65.3 + 26.0	66 335 ×0.014	200 998 ×0.0047	—	—
`int-complex-n4-r6-mu2`	4 / 6	283 183 + 46.5 + 53.5	635 ×0.45	1918 ×0.15	3685 ×0.077	2493 ×3.93
`int-complex-n6-r8-mu2`	6 / 8	2466 1552 + 199 + 713	79 084 ×0.031	235 296 ×0.010	—	17 002 ×0.21
`int-real-n7-r9-muall`	7 / 9	4766 3779 + 333 + 652	438 836 ×0.011	1 316 134 ×0.0036	—	—
`shifted-mixed-n5-r7`	5 / 7	799 487 + 97.9 + 211	15 207 ×0.053	44 780 ×0.018	—	—

Table 5: **All 111 rank $n+2$ cases by n.** Geometric means over the cases of each n of NinjaRS's fast-engine total (width 32) and COLLIER's COLI time (ε^0), and of their ratio, with its range; the same ratio at width 1, against COLI's full Laurent expansion, against DD and against the stock build. The last column is the generic engine's total over the fast engine's (width 32; synthetic and `rn2-int-complex` cases).

n	cases	NinjaRS [µs]	COLI [µs]	÷ COLI	range	÷ COLI (w1)	÷ COLI Laurent	÷ DD	÷ COLI stock	generic ÷ fast
1	12	0.903	1.74	×0.52	×0.22–0.65	×0.64	×0.080	×0.51	×0.62	×11
2	19	9.84	7.95	×1.24	×0.46–1.81	×1.34	×0.33	×1.40	×1.55	×14
3	15	50.2	43.5	×1.15	×0.39–1.55	×1.20	×0.37	×0.66	×1.39	×11
4	19	216	808	×0.27	×0.090–0.45	×0.28	×0.088	×0.17	×0.29	×9.87
5	19	713	14 285	×0.050	×0.027–0.069	×0.051	×0.017	—	×0.053	×8.52
6	19	1940	74 921	×0.026	×0.014–0.032	×0.026	×0.0086	—	×0.027	×7.68
7	8	4968	437 696	×0.011	×0.0095–0.013	×0.012	×0.0038	—	×0.012	×6.83

Table 6: **Retired user-space instructions per phase-space point** (millions; `perf stat -e instructions:u`, the difference of two fixed-work runs of each code, so start-up and recording are excluded). Unlike wall-clock time they do not depend on the core clock, which varies between 1.8 and 3.1 GHz on the shared host. NinjaRS fast engine total at widths 1 and 32; COLLIER finite part (precise COLI and DD, stock COLI). The ratio is NinjaRS (width 32) ÷ COLLIER COLI.

Case	n / r	NinjaRS w1	NinjaRS w32	COLI	DD	COLI stock	÷ COLI
`syn-n1-r3`	1 / 3	0.010	0.009	0.013	0.014	0.011	×0.67
`syn-n2-r4`	2 / 4	0.074	0.071	0.063	0.053	0.052	×1.13
`syn-n3-r5`	3 / 5	0.531	0.525	0.317	0.297	0.277	×1.66
`syn-n4-r6`	4 / 6	1.88	1.88	4.47	4.06	4.18	×0.42
`syn-n5-r7`	5 / 7	5.66	5.87	111	—	106	×0.053
`syn-n6-r8`	6 / 8	16.4	16.4	741	—	691	×0.022
`int-complex-n7-r9`	7 / 9	41.0	41.0	3763	—	3636	×0.011

Time. Up to $n = 3$ the two codes take similar times: per n, NinjaRS's total is ×0.52, ×1.24 and ×1.15 of COLLIER's for $n = 1, 2, 3$ (geometric means, Table 5), between ×0.22 and ×1.81 on single cases. Here COLLIER's tensor integrals are cheap, while NinjaRS spends a median 70 %–76 % of its time in the tensor-numerator expansions of the dense rank $n + 2$ tensor, and only 15 %–23 % in the reduction itself. From $n = 4$ on, COLLIER's cost grows much faster with the rank: NinjaRS takes ×0.27, ×0.050, ×0.026 and ×0.011 of COLLIER's time for $n = 4, \ldots, 7$, where COLLIER needs 438 ms per point at $n = 7$ against NinjaRS's 4.97 ms. A `perf` profile of COLLIER on `syn-n5-r7` and `syn-n6-r8` attributes 69 % and 57 % of its time to building the tensor components from the Lorentz-covariant coefficients (`addtotensortn`

in `BuildTensors.F90`); at rank $n+2$ the number of components and of their covariant structures grows steeply. The retired instructions (Table 6) show the same picture independently of the core clock: from ×1.66 at $n=3$ to ×0.011 at $n=7$. COLLIER's full Laurent expansion costs it ×3.5 of its finite part, so against it NinjaRS gains another factor of about three. DD, where it applies ($n \le 4$), takes ×1.02, ×0.89, ×1.76 and ×1.57 of COLI's time for $n=1,\dots,4$, and the stock build ×0.89 of the precise build's. COLLIER's global cache, which can only reuse integrals within one phase-space point, changes its time by ×1.05, ×1.02, ×1.00, ×0.89, ×0.93, ×0.94 and ×0.53 for $n=1,\dots,7$. The generic engine in binary64 takes ×10 of the fast engine's time, and ×13 for the pure reduction alone. Its numerator here is `case-check`'s numerical expansion of the dense tensor, not the production path of the generic engine, which prepares the numerator symbolically.

Table 7: **Cost of rank $n+2$ against the rank $n+1$ twins** (`rn2-int-complex`, the same kinematics and complex masses; refresh of the table in `docs/rank-n2.md` §7.2, now with the conditioned bases). NinjaRS fast engine: pure reduction (replayed numerator and masters) at width 32, tensor-numerator expansions, masters and total at width 32; COLLIER COLI (ε^0). Each cell gives rank $n+2$ in bold, the rank $n+1$ twin below it and their ratio; here the colour reads rank $n+2$ ÷ rank $n+1$.

n	reduction [µs]	numerator [µs]	masters [µs]	total [µs]	COLLIER COLI [µs]
1	0.145 r = n+1 0.172 ×0.85	0.739 r = n+1 0.230 ×3.21	0.104 r = n+1 0.103 ×1.01	0.989 r = n+1 0.514 ×1.93	1.87 r = n+1 1.22 ×1.53
2	2.69 r = n+1 1.20 ×2.24	7.89 r = n+1 2.33 ×3.38	0.571 r = n+1 0.542 ×1.05	11.1 r = n+1 4.06 ×2.73	8.72 r = n+1 5.89 ×1.48
3	15.6 r = n+1 6.11 ×2.55	42.1 r = n+1 13.4 ×3.14	5.03 r = n+1 4.91 ×1.03	62.6 r = n+1 24.6 ×2.54	47.3 r = n+1 28.6 ×1.65
4	46.3 r = n+1 18.6 ×2.49	160 r = n+1 53.7 ×2.98	53.6 r = n+1 52.3 ×1.02	260 r = n+1 127 ×2.05	595 r = n+1 262 ×2.27
5	103 r = n+1 42.1 ×2.45	492 r = n+1 190 ×2.59	230 r = n+1 226 ×1.01	833 r = n+1 463 ×1.80	14 670 r = n+1 2784 ×5.27
6	198 r = n+1 83.2 ×2.38	1234 r = n+1 541 ×2.28	711 r = n+1 704 ×1.01	2208 r = n+1 1341 ×1.65	75 150 r = n+1 16 882 ×4.45
7	344 r = n+1 152 ×2.26	3032 r = n+1 1321 ×2.30	1616 r = n+1 1645 ×0.98	5018 r = n+1 3105 ×1.62	451 285 r = n+1 107 211 ×4.21

Cost of rank $n+2$. Against their rank $n+1$ twins, the rank $n+2$ lanes cost NinjaRS ×2.2–2.5 in the pure reduction for $n=2,\dots,7$ (`docs/rank-n2.md` measured ×2.1–2.9 at $n=2,\dots,6$ before the basis conditioning, which added 3–6 % instructions; the tadpole at $n=1$: ×0.85) and ×1.6–2.7 in total, where the tensor-numerator expansions (×2.3–3.4) dominate. COLLIER's time grows by ×1.5–5.3 over the same step.

Table 8: **Binary64 accuracy against the high-precision references**, on the tier A and B lanes with a condition of at most 10^6, the lanes on which `case-check` judges binary64 (in brackets: the excluded lanes, tier C, whose published value is COLLIER's own quadruple-precision result or lacks an independent check, and conditions above 10^6, on which no binary64 code is accurate). Scaled difference as `ninjars case-check` computes it: the largest $|x - x_{\mathrm{ref}}|$ over the three Laurent coefficients, divided by the lane's largest reference coefficient. Each cell gives the median and, below it, the maximum. NinjaRS fast and generic engines at the case's own batch width (case-check); COLLIER precise build. The last column counts the lanes on which the fast engine is at least as accurate as COLI.

n	lanes	NinjaRS fast	NinjaRS generic f64	COLI	DD	COLI stock	fast ≤ COLI
1	36	8.0e−16 1.6e−14	2.7e−16 4.8e−15	2.1e−16 1.1e−15	2.4e−16 1.9e−15	2.1e−16 1.1e−15	17 / 36
2	57	1.2e−15 9.1e−12	1.1e−15 2.5e−9	1.3e−15 4.1e−13	9.7e−16 6.7e−14	1.4e−15 2.2e−13	30 / 57
3	45	7.1e−15 2.0e−12	4.3e−15 7.2e−13	2.8e−14 1.9e−10	4.3e−14 3.6e−11	3.7e−14 1.3e−10	28 / 45
4	57	2.8e−14 9.1e−9	1.0e−14 3.4e−9	3.0e−12 9.8e−9	4.1e−12 9.7e−9	2.1e−12 6.8e−9	48 / 57
5	52 (+5)	5.3e−13 3.3e−9	4.2e−13 1.3e−9	1.3e−10 3.5e−7	—	1.0e−10 8.0e−7	45 / 52
6	42 (+15)	5.9e−12 5.4e−9	2.5e−12 5.2e−7	2.1e−10 2.0e−6	—	2.9e−10 2.0e−6	36 / 42
7	9 (+15)	7.1e−12 5.3e−8	5.7e−12 3.5e−8	2.9e−9 3.9e−8	—	2.9e−9 3.9e−8	8 / 9

Accuracy. On the 298 suite lanes that `case-check` judges in binary64 (Table 8), the fast engine's error is at most COLI's on 212 of 298 lanes. Both lose accuracy with n, for different reasons: NinjaRS through cancellations between the cut

contributions and in the subtractions, COLLIER through its Gram and Cayley expansions and the size of its rank $n+2$ tensor reduction. The largest errors for $n = 5, 6, 7$ are 3.3e−9, 5.4e−9, 5.3e−8 for the fast engine and 3.5e−7, 2.0e−6, 3.9e−8 for COLI. On the 19 excluded tier A and B lanes with a condition above 10^6 (all at $n \geq 5$: COLLIER's terms or the new rank $n+2$ terms cancel) neither code is accurate in binary64: up to 1.4e5 for the fast engine and 1.1e9 for COLI.

Table 9: **Binary64 accuracy on the final review's fresh cases** (58 cases, 166 lanes, $n = 1, \ldots, 7$ at rank $n+2$; light-like, equal, mixed, zero and complex masses, μ^2 numerators; references to 60–100 digits). Median and, below, maximum scaled difference, with the number of lanes above 10^{-6}; NinjaRS fast and generic engines (`case-check`), COLLIER precise build.

n	lanes	NinjaRS fast	NinjaRS generic f64	COLI	DD
1	18	8.9e−16 6.7e−15	2.1e−16 1.5e−15	8.0e−17 2.7e−16	8.0e−17 2.7e−16
2	36	8.5e−16 7.5e−13	1.2e−15 6.1e−12	6.0e−16 5.7e−15	5.8e−16 2.9e−15
3	24	4.8e−14 8.9e−10	1.2e−13 8.1e−10	1.9e−14 2.1e−10	1.2e−14 3.7e−11
4	24	6.5e−12 5.9e−5 (3 > 1e-6)	6.5e−12 5.4e−4 (2 > 1e-6)	2.7e−12 1.2e−7	7.6e−13 2.5e−6 (1 > 1e-6)
5	24	6.6e−10 6.7e0 (5 > 1e-6)	1.5e−9 7.1e0 (6 > 1e-6)	2.3e−10 7.2e−4 (2 > 1e-6)	—
6	24	1.4e−9 1.2e8 (8 > 1e-6)	6.2e−10 1.8e8 (8 > 1e-6)	2.9e−9 1.2e0 (4 > 1e-6)	—
7	16	3.0e−10 8.7e6 (4 > 1e-6)	1.8e−10 9.9e7 (4 > 1e-6)	7.7e−10 6.7e−2 (4 > 1e-6)	—

Fresh cases. The final review generated 58 further rank $n+2$ cases (166 lanes) with references to 60–100 digits, including deliberately ill-conditioned light-like kinematics (Table 9). There NinjaRS's binary64 error exceeds 10^{-6} on 21 lanes, all of whose cut contributions cancel by more than 3e6 relative to the result: an instability of integrand reduction in fixed precision that a tensor-integral code does not share, and COLI is accurate to a median 4.8e−9 on these lanes. The generic engine's opt-in cancellation test (`--stability-test`) reports 18 of them as `Unstable`; it leaves 0 lanes with an error above 10^{-5} unflagged and flags 0 lanes whose error is below 10^{-6}. Flagged points can be recomputed in DoubleFloat, which recovers these lanes (final review). COLI in turn exceeds 10^{-6} on 10 lanes, 10 of them with accuracy flag -1.

Table 10: **COLLIER's own accuracy information against its actual error** (tier A and B lanes, precise build). Error flag (negative: an internal error) and accuracy flag (0: target 10^{-8} reached, -1: not reached, -2: critical 10^{-1} not reached), the most negative over the lane's calls; lanes whose actual scaled error exceeds 10^{-8}, how many of those report accuracy flag 0, and how many lanes report a missed target while being accurate to 10^{-8}. The error estimate is $\sum_k |c_k|\ \Delta T(P_k)$ over the numerator coefficients c_k and COLLIER's per-rank absolute error estimates of the tensor components (`TNerr`), divided by the lane scale; the last columns count the lanes whose actual error it covers (an estimate below 10^{-15} counts as 10^{-15}) and give the worst ratio of actual error to estimate.

branch	lanes	errflag < 0	accflag 0 / −1 / −2	error > 1e-8	… with accflag 0	accflag < 0, error ≤ 1e-8	estimate ≥ error	worst error ÷ estimate
COLI	317	0	264 / 53 / 0	44	20	29	303	6.5e3
DD	195	0	177 / 18 / 0	0	0	18	185	1.0e3

COLLIER's own accuracy information (Table 10) is a warning, not a bound: COLI reports a missed target ($\sigma_{\text{acc}} = -1$) on 53 of 317 lanes, but on 20 of the 44 lanes whose actual error exceeds 10^{-8} it reports the target as reached, and its error estimate falls short of the actual error on 14 lanes, by up to ×6.5e3. The quadruple-precision builds showed the same in the reference harness (`validation/rank-n2/README.md`, finding 1).

Table 11: **`ninjars case-check` on the 111 rank $n+2$ cases** (suite and synthetic, 333 lanes), as pass / fail / report-only / known failure per lane, with the tolerance policy of `docs/rank-n2.md` §8.3: in binary64 a budget from original Ninja's error on the rank twins (30× for the fast engine, 100× for the generic engine) plus the references' uncertainty; at high precision three times the uncertainty. Report-only lanes are tier C, conditions above 10^6 in binary64, DoubleFloat and Float(128), tier-B uncertainties taken from the generic engine's own twins at high precision, and lanes whose smallest new rank $n+2$ term lies below the tolerance.

n	lanes	fast (f64)	generic f64	generic DoubleFloat	generic Float(128)	generic Float(256)
1	36	36 / 0 / 0 / 0	36 / 0 / 0 / 0	36 / 0 / 0 / 0	36 / 0 / 0 / 0	36 / 0 / 0 / 0

n	lanes	fast (f64)	generic f64	generic DoubleFloat	generic Float(128)	generic Float(256)
2	57	55 / 0 / 2 / 0	55 / 0 / 2 / 0	57 / 0 / 0 / 0	57 / 0 / 0 / 0	49 / 0 / 8 / 0
3	45	44 / 0 / 1 / 0	44 / 0 / 1 / 0	45 / 0 / 0 / 0	45 / 0 / 0 / 0	45 / 0 / 0 / 0
4	57	53 / 0 / 4 / 0	52 / 0 / 5 / 0	57 / 0 / 0 / 0	56 / 0 / 0 / 1	49 / 0 / 8 / 0
5	57	47 / 0 / 10 / 0	47 / 0 / 10 / 0	52 / 0 / 5 / 0	52 / 0 / 5 / 0	50 / 0 / 7 / 0
6	57	32 / 0 / 25 / 0	32 / 0 / 25 / 0	41 / 0 / 16 / 0	40 / 0 / 16 / 1	41 / 0 / 16 / 0
7	24	9 / 0 / 15 / 0	9 / 0 / 15 / 0	9 / 0 / 15 / 0	8 / 0 / 15 / 1	15 / 0 / 9 / 0
largest checked		5.3e−8	5.2e−7	1.4e−11	1.4e−11	7.0e−12
largest tier A		5.3e−8	3.5e−8	1.3e−24	1.2e−29	5.9e−30

Correctness of NinjaRS at rank $n + 2$ is checked lane by lane by `ninjars case-check` (Table 11): no lane fails in any of the five runs, which reproduce `docs/rank-n2.md` §8.3. **The COLLIER driver** reproduces the binary64 COLLIER 1.2.9 diagnostics stored with the references (exact decimal inputs and a 90-digit contraction there, binary64 here): on the 220 COLI lanes with an error above 10^{-14} the ratio of the errors has median 1.000, and the 5 lanes that differ by more than a factor of two are μ^2 lanes, whose pieces this harness rounds to binary64 before the contraction. COLLIER 1.2.10 changes, against 1.2.9, complex-mass C_0 root prescriptions and two error-estimate guards. Building and running COLLIER showed one defect in the reference harness and one cost to know about: its driver aggregated COLLIER's accuracy flag with `max` from 0 and so never reported a negative flag (the references do not use it), and every change of a regulator or scale parameter runs DD's machine-precision loop (`DDsetparam`), also in COLI mode.

6 How the numbers are measured

Test cases. Seven physical one-loop diagrams with a top-quark loop, generated independently by FeynKit (NinjaRS) and MadGraph5_aMC@NLO 3.7.0 (MadLoop), and matched graph by graph: the internal masses agree exactly, the propagator invariants to binary64 rounding, and the integrated results up to a constant normalization over all phase-space points. Each diagram is evaluated at three phase-space points (exact rational/radical momenta rounded once to binary64), one fixed helicity configuration, zero widths and $\mu^2 = 100^2$ GeV² (125^2 GeV² for $gg \to H$). Twenty synthetic topologies ($n = 2, \ldots, 6$ propagators, ranks $n - 1$, n and $n + 1$, random real kinematics with time-like and space-like invariants, real and complex masses, dense random tensor numerators) complete the suite.

NinjaRS. `ninjars pipeline-bench` builds the physical numerator from the DOT graph and the UFO-style model (Feynman rules, wavefunctions, colour projection, Dirac and Lorentz algebra, exact polynomial in the loop momentum), plans and compiles its coefficient heads with SymJIT O2 and then times, per batch of phase-space points: the numerator preparation (wavefunctions, couplings, coefficient heads, Ninja tensor layout and propagator data); the pure reduction (`ninjars::fast` with the numerator outputs and master values of the same evaluation replayed at O(1) cost); the tensor-numerator expansions (live tensor numerator minus pure reduction); the master integrals (the recorded calls replayed through oneloopmaster's native code); and the uninstrumented total. At the tabulated batch width the components add up to the total within 6 %. `ninjars case-bench` performs the same breakdown on the shared bench-case files, which contain exactly the propagators and tensor coefficients given to original Ninja.

MadLoop pipeline. Optimized standalone MadLoop output (`loop_optimized_output`), restricted to the same single loop diagram, Ninja as the only reduction library, no stability rotations, linked against original Ninja 1.2.1 (`g++ -O3 -fcx-fortran-rules`, integral cache disabled) and OneLOop 3.6 (`gfortran -O3`). An instrumented build splits each evaluation into MadLoop's loop-coefficient construction, its Ninja interface (tensor copy, momentum inversion, S-matrix), Ninja's tensor-numerator callbacks, the OneLOop master integrals and the rest of Ninja's entry point (the reduction, including Ninja's per-call setup). The numerator column sums the first three. Replaying MadLoop's exact Ninja inputs in the C++ driver below reproduces the master and tensor-numerator times within 5 %; the instrumented reduction is 14–17 % above Ninja's pure reduction measured there (replayed numerator and masters, as for NinjaRS), which is the reference of Table 2 and of gate (a). The table uses the faster MadLoop variant for every quantity: `GLOBAL_FLAG=-O3` for evaluation and the default flags for compilation time.

Identical-input comparison. The C++ driver feeds original Ninja the same bench cases (`Amplitude<ComplexMasses>`, `TensorNumerator`, explicit S-matrix) and replays recorded numerator outputs and OneLOop results to isolate each component, exactly like the NinjaRS harness. Every replay reproduces the direct evaluation bit for bit.

Environment. A shared AMD EPYC 9754 host; every timed process, NinjaRS and the references alike, runs single-threaded and pinned to the same core (CPU 250; the arbitrary-precision runs of Section 4 used CPU 245). The reference timings are from 2026-10-18, the NinjaRS timings from 2026-10-19 (Symbolica e679e8c1), measured with oneloopmaster 3023fa6 (the arbitrary-precision timings with the pinned 09b2c74). 149ddff and 09b2c74 change no binary64 evaluated call (every benchmark case and example gives bit-identical binary64 and double-double results with all three, and the native masters return the same bits on every recorded call), so they stand. 09b2c74 rounds the interpreted D_0 differently, by at most 56 units in the last place at 700 and 1000 bits; the results of Section 4 are computed with 09b2c74. References timed a day earlier on another core (CPU 29) agree within about 1 %, except OneLOop on the $n = 5$ cases (5–8 % faster there), which moves no gate (`validation/benchmark/cpu-crosscheck.md`). Rust 1.98 (`--profile perf`: opt-level 3, one codegen unit for the hot crates), GCC and gfortran 15.3, no `-march`/`-ffast-math` on either side. Timings are medians of at least seven blocks of at least 20 ms; independent repeated runs agree within 1–4 %, sub-microsecond components within 10–20 %.

7 What is new in NinjaRS

NinjaRS reimplements the integrand-reduction method of Ninja [1], [6] in Rust and embeds it in an end-to-end pipeline built on shared physics and algebra libraries. The original stack used for comparison — MadLoop's numerator [7], Ninja's reduction and OneLOop's master integrals [8] — is a set of independently written Fortran and C++ codes connected through a tensor-numerator interface [9]. NinjaRS replaces each component while keeping Ninja's semantics and results.

7.1 One exact source, from Feynman graph to integrand

Physical numerators are prepared by FeynKit from Feynman-graph DOT files and a UFO-style model: Feynman rules, external wavefunctions, colour projection and Dirac/Lorentz algebra are applied symbolically with Spenso/Idenso and Symbolica. The result is an **exact** polynomial in the loop momentum (and μ^2) whose coefficients are exact expressions in the external kinematics and model parameters. Its distinct coefficient "heads" are deduplicated, optionally factored to remove numerical cancellations, and compiled together into a single multi-output SymJIT evaluator at optimization level O2. Per phase-space point NinjaRS evaluates these heads once and scatters them into Ninja's tensor layout with a precomputed, allocation-free map; MadLoop instead rebuilds its loop polynomial through an open-loop recursion over loop wavefunctions at every point.

Because the polynomial is exact, NinjaRS knows the **true** rank of each numerator. The odd-γ trace removes the q^5 terms of the top-quark pentagons of $gg \to HHH$ and the q^3 terms of the triangles, which NinjaRS therefore reduces at rank 4 and 2, while MadLoop declares ranks 5 and 3 and drives Ninja through more expensive expansions (rank $r = n$ instead of the true $r = n - 1$).

7.2 Batching throughout the pipeline

Every NinjaRS interface — numerator coefficients, cut-numerator callbacks, reduction and master integrals — processes a batch of phase-space points together. Callbacks receive one compact batch per cut instead of one call per point and cut; master integrals are requested once per integral family for all cuts and points of a batch. Failures are isolated per phase-space lane: a lane failing a stability check or a master evaluation keeps its previous result while its neighbours are accumulated.

7.3 Generic numerical domains and derivatives

The generic reduction engine is written once for any Symbolica numerical domain: binary64, double-double (`DoubleFloat`), arbitrary-precision `Float` with tracked significant precision, and Symbolica's Dual numbers for first, higher and mixed derivatives with respect to kinematic or model parameters. Precision rescue and derivatives therefore need neither a separate quadruple-precision build nor numerical differencing. Its arithmetic kernels are generated mechanically from Symbolica instruction graphs.

7.4 A dedicated binary64 engine

For ordinary double-precision evaluation, `ninjars::fast` is a monomorphic specialization of the same Laurent reduction, batched over phase-space points, with plain complex arithmetic, precomputed topology data and allocation-free workspaces. It covers every cut level, any number of propagators and ranks up to $n + 1$, with Ninja's default options, skip rules and stability checks, and reproduces original Ninja's results including the split into cut-constructible and rational parts. It is the engine measured in Table 1 and Table 2.

7.5 Master integrals from exact expressions: oneloopmaster

oneloopmaster encodes OneLOop's scalar integrals as exact Symbolica expressions, including analytic-continuation repairs, and generates direct Rust implementations from them. One source therefore serves binary64, double-double and arbitrary precision (and Dual numbers through the interpreter), with lazy conditional branches preserved. The bubble and tadpole tensor coefficients are native ports of OneLOop's routines. The corrections to the scalar integrals found while benchmarking (Section 3) were made in the exact expressions, so all of these routes, and the hand-written binary64 C_0/D_0 kernels transcribed from them, carry them. Replayed call by call on the master calls original Ninja makes in the benchmark, the native masters take 0.25–0.86× the time of OneLOop.

8 Algorithmic improvements

The speed-ups in Table 2 come from changes in how the work is organized, not from different mathematics: NinjaRS and Ninja agree to 10^{-12} or better on the physical diagrams, and the larger deviations of some synthetic cases are inherited binary64 conditioning (Section 3).

- **Topology precomputation.** Cut partitions, complements, parent lists for the coefficient-level subtractions and tadpole reference candidates are built once per topology. Ninja allocates its cut containers and rescans sub-partition relations at every phase-space point.
- **Batched callbacks and masters.** One numerator call per cut (both orientations of a triangle share it) and one master call per family per batch, instead of per point and cut; all master requests of all cuts and points are collected before a single contraction pass.
- **Truncated-jet tensor expansions.** Ninja's `TensorNumerator` materializes symmetric tensor powers of the cut vectors (e_3^r, $e_3^{r-1}a$, ...) and contracts them with the numerator. NinjaRS instead evaluates the homogenized numerator $t^{-R}N(q)$ as one transposed Horner sweep in $u = \frac{1}{t}$ truncated at the requested order, with exact polynomial dependence on μ^2 and x: every tensor coefficient is touched once per expansion. This makes the tensor numerator 1.9–2.4× faster than Ninja's on the physical diagrams (1.7–2.6× over all cases).
- **Rank-specialized subtractions.** Ninja's FORM-generated coefficient-level subtractions are translated mechanically, and the parent coefficients that vanish identically for the given rank are folded away at compile time (Ninja evaluates every term and discards some afterwards).
- **Shared per-propagator data.** Products of cut-basis vectors with the propagator shifts and their reciprocals are computed once per cut and lane and reused by the Laurent divisions of both orientations and by the subtractions; each uncut denominator costs one reciprocal.
- **Plain complex arithmetic.** Rust's complex multiply and divide have no NaN/infinity recovery, which is what `-fcx-fortran-rules` gives C++; we measure original Ninja with that flag so the comparison is not inflated by `__muldc3`/`__divdc3` calls (worth 20 % on Ninja's side).
- **True numerator rank.** As described above, the exact numerator avoids the inflated ranks MadLoop declares, which makes Ninja's tensor expansions 3.7× more expensive for the pentagon (81 µs at rank 5 against 22 µs at rank 4) and about 5× for the triangles in the MadLoop pipeline.
- **Single-dispatch master kernels.** The generated oneloopmaster code evaluated each Laurent order of C_0 and D_0 separately, repeating the channel and sector selection three times. Hand-transcribed binary64 kernels for the dominant sectors (three-mass triangles, contour boxes and the four-mass Denner–Nierste–Scharf box) perform one dispatch per call, never evaluate inactive branches, and keep the exact expressions as their validated reference; they are 2–3.5× faster than the generated code and, on the physical call streams, 1.5–1.8× (boxes) and 1.9–3.6× (triangles) faster than OneLOop.
- **Fast binary64 dilogarithm and native tensor bubbles.** oneloopmaster's binary64 dilogarithm now uses a dedicated series kernel (17–110 ns per call instead of an MPFR detour; contributed to Symbolica as PR #89), and the bubble tensor coefficients $B_0, B_1, B_{11}, B_{111}$ come from one native evaluation sharing the roots and logarithms (a binary64 specialization of oneloopmaster's port of OneLOop's bubble routines, validated against the exact expressions and Feynman-parameter integrals), 3.8–12× faster than OneLOop's `olo_b11` on the physical call streams.
- **On-shell thresholds in one place.** Exact-zero dispatch in oneloopmaster is combined with OneLOop's on-shell threshold as Ninja configures it, applied once to every master request, and MadLoop's on-shell treatment of the kinematic matrix is reproduced when propagator data are prepared.

Further possible improvements. The per-point arithmetic of the reduction is scalar; vectorizing it across phase-space points (structure-of-arrays) is the natural next step. Master requests could be deduplicated across diagrams of a process, and the box and triangle masters could select the cheapest equivalent argument ordering, as OneLOop does

internally. For the generic engine, staged fusion of the reduction stages up to master weights reduces the instruction count of an $n = 3$, $r = 3$ reduction from 1,884 to 1,071 operations in an exact inventory.

References

[1] T. Peraro, "Ninja: Automated integrand reduction via Laurent expansion for one-loop amplitudes," *Comput. Phys. Commun.*, vol. 185, 2014.

[2] A. Denner, S. Dittmaier, and L. Hofer, "COLLIER: a fortran-based Complex One-Loop LIbrary in Extended Regularizations," *Comput. Phys. Commun.*, vol. 212, 2017.

[3] A. Denner and S. Dittmaier, "Reduction of one-loop tensor 5-point integrals," *Nucl. Phys. B*, vol. 658, 2003.

[4] A. Denner and S. Dittmaier, "Reduction schemes for one-loop tensor integrals," *Nucl. Phys. B*, vol. 734, 2006.

[5] A. Denner and S. Dittmaier, "Scalar one-loop 4-point integrals," *Nucl. Phys. B*, vol. 844, 2011.

[6] P. Mastrolia, E. Mirabella, and T. Peraro, "Integrand reduction of one-loop scattering amplitudes through Laurent series expansion," *JHEP*, vol. 1206, 2012.

[7] V. Hirschi, R. Frederix, S. Frixione, M. V. Garzelli, F. Maltoni, and R. Pittau, "Automation of one-loop QCD corrections," *JHEP*, vol. 1105, 2011.

[8] A. van Hameren, "OneLOop: for the evaluation of one-loop scalar functions," *Comput. Phys. Commun.*, vol. 182, 2011.

[9] V. Hirschi and T. Peraro, "Integrand reduction of one-loop amplitudes with a tensor numerator," *JHEP*, vol. 1606, 2016.